# SiftTSP: A Geometric Sandclock-Sift Heuristic for Euclidean TSP

Norbert Nopper

**Abstract**

We present **SiftTSP**, a deterministic polynomial-time heuristic for the Euclidean Travelling Salesman Problem in the partitioning lineage of Karp [8]. The algorithm partitions the city set by recursive geometric bisection into $2^d$ sections, solves small sections by brute force and larger ones by a complementary pair of greedy rules (min-max / max-min), and reconnects the section paths by exhaustive search over orderings and orientations. The angular dimension of the search is covered by two complementary sweeps — a uniform *mill* (breadth) and a damped fan-shaped *sieve* (depth) — coordinated by a *sandclock* depth schedule $1 \to d_{\max} \to 1$ and an 8-state Phase 4 loop over (section-mode, mill-direction, ordering). The contribution is the specific orchestration of these primitives (§1.5), not a new theoretical guarantee.

For any fixed parameters the algorithm runs in $O(n^2)$ time (Theorem 1). SiftTSP is **not exact**: we exhibit an explicit 12-point Euclidean instance (`counterexample.py`, `rings-12`) on which the algorithm misses the Held–Karp optimum at every tested depth, with gap +3.24% at $d = 3$. The structural reason is identified as *decomposition irrecoverability* (§2.3): no fixed family of geometric decompositions can recover an optimal tour that interleaves cities across components of every decomposition in the family. Consequently this work makes **no claim** of resolving $\mathcal{P}$ versus $\mathcal{NP}$ [1, 2, 3], and offers no approximation ratio (unlike Christofides [9] for metric TSP or Arora [12] for Euclidean TSP).

On a battery of 14 instances ($n \in \{12, 14, 16\}$) including zigzag, comb, interleaved, concentric-ring, and uniform configurations, SiftTSP matches Held–Karp on most structured inputs and beats 2-opt on the rotationally symmetric `rings-12` and `rings-14` instances despite not being exact.

**Keywords:** Travelling Salesman Problem, Polynomial-Time Heuristic, Euclidean TSP, Geometric Subdivision, Mill-Sieve Search.

# 1 Algorithm

## 1.1 Setting

Given $n$ cities $C = \{c_1, \ldots, c_n\} \subset \mathbb{R}^2$, a *tour* is a cyclic permutation $\pi$ of $C$, and its length is $L(\pi) = \sum_i \|c_{\pi(i)} - c_{\pi(i+1)}\|_2$ (indices mod $n$). Euclidean TSP asks for an $L$-minimum tour. The decision form of general (metric) TSP is NP-Complete by Karp's reduction from Hamiltonian Cycle [5]; Papadimitriou [6] established that the restriction to Euclidean instances in the plane is also NP-Complete, hence the optimisation form is NP-Hard. The exact Held–Karp DP [7] runs in $O(n^2 \cdot 2^n)$ time and is feasible only for small $n$. SiftTSP is a polynomial-time *heuristic* — it is not exact, see §2.3.

## 1.2 Parameters and Phases

SiftTSP is controlled by four constants treated as fixed:

| Symbol | Meaning |
|---|---|
| $d_{\text{ceiling}}$ | Maximum recursive subdivision depth |
| $m_{\text{ceiling}}$ | Mill (breadth) cap: third-steps per direction |
| $s_{\text{ceiling}}$ | Sieve (depth) cap: damped halvings around mill best |
| $\tau$ | Brute-force threshold: sections with $\leq \tau$ cities are solved exactly |

The algorithm is structured in four phases:

- **Phase 1 (max).** A worst-case greedy-longest tour, computed once, fixes an initial upper bound $B_0 \geq L^\star$ that the rest only lowers.
- **Phase 2 (sift).** At each depth $d$ on the sandclock schedule $1, 2, \ldots, d_{\max}, \ldots, 2, 1$ and at each angle $\theta$ in the current mill+sieve schedule, the city set is partitioned by recursive median bisection along **each** of four axis-pairs (each pair: two perpendicular axes from the rotated frame at angle $\theta$ — the primary axis governs the top-level split, the secondary axis the next level, alternating thereafter). The four pairs are taken from {vertical, /, horizontal, \}. Each axis-pair yields a partition into $2^d$ sections, evaluated independently. Sections of size $> \tau$ are solved by a greedy section solver in one of two complementary modes (selected per sweep by Phase 4):
  - **min-max**: at each step, append the city minimising the running *maximum* edge (favouring smooth paths);
  - **max-min**: at each step, append the city maximising the running *minimum* edge (a complementary ordering).
- **Phase 3 (min).** Sections of size $\leq \tau$ are solved exactly by brute force. The $2^d$ section paths are reconnected by exhaustive search over $(2^d - 1)! \cdot 2^{2^d - 1}$ orderings and orientations.
- **Phase 4 (loop).** An 8-state super-cycle drives the angular search.

Both greedy rules are seeded at the first city in the section's input ordering and extend the path one vertex at a time; consequently SiftTSP is deterministic but its output depends on the order in which the cities are presented (the partitioning, the section orderings inside each partition, and the brute-force fall-backs all inherit this order-dependence).

The angular dimension uses two complementary sweeps. The **mill** is a uniform third-step rotation across $[0, \pi/2)$ in both directions (breadth). The **sieve** is a damped forth-and-back oscillation around the best $(\theta^\star, d^\star)$ found by the mill, with amplitudes $\frac{s_0}{2}, \frac{s_0}{4}, \ldots, \frac{s_0}{2^s}$ where $s_0 = (\pi/2^{d^\star})/3$ (depth). Mill and sieve refine the *same* angular axis at different scales; together they sift the search through both wide and narrow neighbourhoods of every depth.

### 1.3 The Phase 4 Super-Cycle

Each sandclock sweep is parameterised by a tuple $(d_{\max}, m, s, \mu, \delta)$, where $\mu$ is the section-solver mode (min-max / max-min) and $\delta$ is the angular-sweep direction (CW / CCW). The four configurations $(\mu, \delta)$ may be visited in two orderings — *mode-first* (MD) or *direction-first* (DM) — giving an 8-state super-cycle:

| $k$ | $\mu$ | $\delta$ | ordering |
|---|---|---|---|
| 0 | max-min | CW | MD |
| 1 | min-max | CW | MD |
| 2 | max-min | CCW | MD |
| 3 | min-max | CCW | MD |
| 4 | max-min | CW | DM |
| 5 | max-min | CCW | DM |
| 6 | min-max | CW | DM |
| 7 | min-max | CCW | DM |

The order of visitation matters because the mill cap $m$ and sieve cap $s$ double *only on improvement*; the sequence of $(\text{config}, m, s)$ triples ever evaluated depends on the order.

**Transition rule.** After a sweep at position $k$:

1. If the *mill* improved the running best, $m \leftarrow 2m$.
2. If the *sieve* improved the running best, $s \leftarrow 2s$.
3. Advance $k \leftarrow (k+1) \bmod 8$.
4. If 8 consecutive sweeps at the same $(d_{\max}, m, s)$ all fail, bump $d_{\max} \leftarrow d_{\max} + 1$, reset $m, s, k$ to $1, 1, 0$, and restart Phase 2.

**Termination.** For fixed parameters $(d_{\text{ceiling}}, m_{\text{ceiling}}, s_{\text{ceiling}})$, the candidate tours examined at any sweep are determined by the discrete configuration $(d_{\max}, m_{\text{eff}}, s_{\text{eff}}, \mu, \delta, k \bmod 8)$, drawn from an $O(1)$-sized set whose size depends only on the parameters and not on $n$. The running best $L$ is updated only on *strict* improvement, so each improving sweep contributes a previously unseen, strictly shorter candidate; the total number of improving sweeps is therefore bounded by the size of this candidate set, hence $O(1)$. Each non-improving sweep increments `no_improve`; reaching 8 breaks the inner `while`. Between any two consecutive improvements there are at most 7 non-improving sweeps, and at most 8 non-improving sweeps follow the last improvement, so the total number of sweeps per $d_{\max}$ is also $O(1)$. The outer `for` loop runs over $d_{\max} = 1, \ldots, d_{\text{ceiling}}$, so the algorithm terminates after $O(1)$ sweeps overall (for fixed parameters). The returned tour has length monotone non-increasing along the execution trace and is bounded below by 0.

## 1.4 Pseudocode

```
SUPERCYCLE := concat(
    [(MaxMin,CW), (MinMax,CW), (MaxMin,CCW), (MinMax,CCW)],   # ordering MD
    [(MaxMin,CW), (MaxMin,CCW), (MinMax,CW), (MinMax,CCW)]    # ordering DM
)                                                              # length 8

procedure SiftTSP(C, d_ceiling, m_ceiling, s_ceiling, tau):
    best <- WorstCaseTour(C); best_len <- Length(best)         # PHASE 1
    for d_max = 1 .. d_ceiling do                              # PHASE 4
        m, s, k, no_improve <- 1, 1, 0, 0
        while m <= m_ceiling or s <= s_ceiling do
            (mode, dir) <- SUPERCYCLE[k]
            (best, best_len, mill_imp, sieve_imp)
              <- SandclockSweep(C, d_max, min(m,m_ceiling),
                                min(s,s_ceiling), tau, mode, dir,
                                best, best_len)
            k <- (k+1) mod 8
```

```
            if mill_imp or sieve_imp then
                no_improve <- 0
                if mill_imp  and m <= m_ceiling then m <- 2*m
                if sieve_imp and s <= s_ceiling then s <- 2*s
            else
                no_improve <- no_improve + 1
                if no_improve >= 8 then break
    return best

procedure SandclockSweep(C, d_max, m, s, tau, mode, dir, best, best_len):
    mill_imp, sieve_imp, theta*, d* <- false, false, 0, d_max
    for d in [1,2,..,d_max,..,2,1] do                          # PHASE 2 mill
        for theta in MillAngles(d, m, dir) do
            (best, best_len, hit) <- EvalAt(C, d, theta, tau, mode, ...)
            if hit then mill_imp, theta*, d* <- true, theta, d
    s0 <- (pi / 2^d*) / 3                                      # PHASE 2 sieve
    for j = 0 .. s-1 do
        delta <- s0 / 2^(j+1)
        for sigma in (+1, -1) do
            (best, best_len, hit) <- EvalAt(C, d*, theta* + sigma*delta, ...)
            if hit then sieve_imp, theta* <- true, theta* + sigma*delta
    return (best, best_len, mill_imp, sieve_imp)

procedure EvalAt(C, d, theta, tau, mode, best, best_len):
    hit <- false
    for (axis_p, axis_s) in AxisPairs(theta) do                # 4 pairs
        sections <- Subdivide(C, [axis_p, axis_s], d)
        paths <- []
        for S in sections do
            if |S| <= tau then paths <- paths + [BruteForce(S)]      # PHASE 3
            elif mode = MinMax then paths <- paths + [MinMax(S)]     # PHASE 2
            else                    paths <- paths + [MaxMin(S)]
        T <- BruteForceConnect(paths)
        if Length(T) < best_len then
            best_len, best, hit <- Length(T), T, true
    return (best, best_len, hit)
```

The reference implementation is `SiftTSP.py`; entry point `sift_tsp`.

### 1.5 Prior Art and Contribution

SiftTSP belongs to the family of **geometric-partitioning heuristics** for Euclidean TSP, whose canonical antecedent is Karp's 1977 probabilistic analysis of partitioning algorithms [8]. In that scheme the plane is divided into rectangles, each sub-instance is solved (optimally or heuristically), and the sub-tours are reconnected. Later geometric heuristics, surveyed by Bentley [13], explore strip, space-filling-curve, and recursive variants. Arora's PTAS [12] is the theoretical apex of geometric methods, achieving a $(1+\varepsilon)$ approximation in polynomial time but with constants that make it impractical at small $n$. State-of-the-art *practical* solvers — LKH [11], built on Lin–Kernighan [10], and Concorde [4] — are not partitioning-based and dominate empirically on TSPLIB-scale instances.

Against this backdrop, **SiftTSP makes no theoretical claim that extends prior art.** Its complexity bound ($O(n^2)$) is no better than several classical heuristics; it offers no approximation ratio (unlike Christofides [9] for metric TSP or Arora [12] for Euclidean TSP); and it is empirically not exact (§2.3). What it offers is an *engineering recipe* combining:

1. a **sandclock depth schedule** $1 \to d_{\max} \to 1$ that visits every depth twice per sweep, once on the way up and once on the way down;

2. a **two-scale angular search** pairing a uniform "mill" (breadth) with a damped "sieve" (depth) around the mill's best angle;

3. an **8-state Phase 4 super-cycle** that systematically permutes section-solver mode (min-max / max-min), angular sweep direction (CW / CCW), and visit ordering (MD / DM), with independent doubling caps on mill and sieve that grow only on improvement.

To the author's knowledge this specific orchestration is new, but each individual primitive (recursive median bisection, exhaustive sub-tour chaining, greedy section solvers, rotational axis search) is standard. We therefore frame SiftTSP as a **deterministic, reproducible, and falsifiable** partitioning heuristic — and ship a concrete witness to its non-exactness (§2.3) rather than leave exactness as an open question.

### 1.6 Complexity

Treat $d_{\text{ceiling}}, m_{\text{ceiling}}, s_{\text{ceiling}}, \tau$ as fixed constants independent of $n$.

**Lemma 1 (Subdivide).** *Algorithm* `Subdivide` *runs in $O(n \log n)$ time and produces at most $2^d$ sections of size $\lceil n/2^d \rceil$ or $\lfloor n/2^d \rfloor$.*

*Proof.* The recursion has depth $d$. At each level the cities in each sub-list are sorted once along the current axis and split at the median, so each level does $O(n \log n)$ comparison work in total. With $d$ fixed, total work is $O(n \log n)$. Median splits keep section sizes balanced. □

**Lemma 2 (Section solving).** *Let $k = 2^d$ and assume the recursive median bisection of Lemma 1, so every section has size in $\{\lfloor n/k \rfloor, \lceil n/k \rceil\}$. Processing all sections takes $O(1)$ time when $\lceil n/k \rceil \leq \tau$ (every section is brute-forced), and $O(n^2/k)$ time otherwise; the mixed regime (some brute-forced, some greedy) is bounded by the larger.*

*Proof.* Brute force on a section of $\leq \tau$ cities is $O(\tau!) = O(1)$, and there are at most $k = O(1)$ such sections, giving $O(1)$ total. For the greedy branch, each section has size $s_i \leq \lceil n/k \rceil$ by balanced bisection, and each greedy solver (`minmax_path`, `maxmin_path`) does an $O(s_i)$ scan at each of its $s_i$ steps, for $O(s_i^2)$ total per section. Summing, $\sum_i s_i^2 \leq k \cdot (\lceil n/k \rceil)^2 = O(n^2/k)$. □

**Lemma 3 (Connection).** *For fixed $d$ (hence fixed $k = 2^d$),* `BruteForceConnect` *runs in $O(n)$ time.*

*Proof.* There are $(k-1)!$ orderings and $2^{k-1}$ flip vectors (both $O(1)$ for fixed $d$), and each candidate tour is scored in $O(n)$. □

**Lemma 4 (Phase 1).** *The greedy-longest construction in the reference implementation runs in $O(n^2)$ time.*

*Proof.* At each of $n$ steps it scans the remaining unvisited cities to find the farthest from the current endpoint and removes it from a list, both $O(n)$ operations. □

**Theorem 1 (Time complexity of SiftTSP).** *For fixed parameters, SiftTSP runs in $O(n^2)$ time on every input.*

*Proof.* Phase 1 costs $O(n^2)$ (Lemma 4). Each `EvalAt` invocation loops over the four axis-pairs (§1.4), each costing $O(n \log n)$ for `Subdivide` (Lemma 1), $O(n^2/k)$ for section solving (Lemma 2; $O(1)$ in the bounded-$n$ all-exact regime), and $O(n)$ for `BruteForceConnect` (Lemma 3); together $O(n^2)$ per `EvalAt`. Because $d_{\max} \leq d_{\text{ceiling}}$ throughout the run, the number of `EvalAt` invocations per `SandclockSweep` is $O(d_{\text{ceiling}} \cdot m_{\text{ceiling}} + s_{\text{ceiling}}) = O(1)$. The total number of sweeps in Phase 4 is $O(1)$ for fixed parameters by the termination argument of §1.3 (the per-sweep configuration is drawn from an $O(1)$-sized set and the running best strictly decreases along improvements).Multiplying gives $O(n^2)$. □

**Corollary 1.** *SiftTSP is a polynomial-time algorithm for all fixed parameters.* Polynomiality is a property of a *heuristic*: §2.3 shows it is not exact in general.

**Remark (sharper bounds in sub-regimes).** Phase 1 dominates the worst-case bound. If Phase 1 were replaced by an $O(n \log n)$ implementation (for example via a $k$-d tree for farthest-point queries), and if $n$ is small enough that every section is brute-forced ($\lceil n/2^{d_{\text{ceiling}}} \rceil \leq \tau$), the overall complexity would drop to $O(n \log n)$. With fixed parameters that regime is reached only for $n \leq \tau \cdot 2^{d_{\text{ceiling}}}$, that is, $n$ bounded by a constant, in which case the entire algorithm is trivially $O(1)$. We therefore report the honest worst-case bound $O(n^2)$.

# 2 Experiments

## 2.1 Setup

All experiments use the Euclidean distance metric. Optimal tours are computed by the exact Held–Karp DP [7] (feasible up to $n \approx 20$). We compare SiftTSP (`SiftTSP.py`, entry point `sift_tsp`) against:

- **Held–Karp**: exact $O(n^2 \cdot 2^n)$ DP — ground-truth optimum.
- **Nearest-Neighbor (NN)**: classic $O(n^2)$ greedy construction.
- **2-opt**: 2-opt local search starting from the NN tour.

SiftTSP is run with $\tau = 6$ and $d_{\max} \in \{1, 2, 3\}$. The optimality gap is $(L/L^\star - 1) \times 100\%$ where $L^\star$ is the Held–Karp optimum.

Reproduce with:

```
$env:PYTHONIOENCODING="utf-8"; python -m tests.adversarial   # full battery
$env:PYTHONIOENCODING="utf-8"; python counterexample.py      # canonical counterexample
```

**Reproducibility.** All experiments run under CPython 3.11 (no third-party numerical libraries). The algorithm is deterministic given $(C, d_{\text{ceiling}}, \tau, m_{\text{ceiling}}, s_{\text{ceiling}})$ and the city input ordering, with no randomised tie-breaking. Random instance families use `random.Random` with the following fixed seeds (`tests/adversarial.py`): `zigzag` = 0, `interleaved` = 1, `comb` = 2, `rings` = 3, `uniform-12/14/16` = 42/43/44. The deterministic instances (`rings-12` of `counterexample.py`) use literal coordinates listed in the source file.

## 2.2 Adversarial Battery

Instance families designed to probe geometric-bisection weaknesses:

- **zigzag**-$n$: $n$ points alternating across a vertical axis.
- **interleaved**-$n$: two clusters with interleaved indices.
- **comb**-$n$: a spine with vertical teeth.
- **rings**-$n$: two concentric rings — no axis is preferred.
- **uniform**-$n$: uniform random control.

**Table 1.** Optimality gap (%) — **bold = exact**, *italic = sub-optimal.*

| Instance | $n$ | NN | NN+2-opt | $d$=1 | $d$=2 | $d$=3 |
|---|---|---|---|---|---|---|
| zigzag-12 | 12 | **0.00** | **0.00** | **0.00** | **0.00** | **0.00** |
| zigzag-14 | 14 | 28.84 | **0.00** | **0.00** | **0.00** | **0.00** |
| zigzag-16 | 16 | **0.00** | **0.00** | **0.00** | **0.00** | **0.00** |
| zigzag-tight-12 | 12 | **0.00** | **0.00** | **0.00** | **0.00** | **0.00** |
| zigzag-wide-14 | 14 | 19.52 | **0.00** | **0.00** | **0.00** | **0.00** |
| interleaved-12 | 12 | 38.09 | **0.00** | **0.00** | **0.00** | **0.00** |
| interleaved-14 | 14 | **0.00** | **0.00** | *31.41* | **0.00** | **0.00** |
| comb-12 | 12 | **0.00** | **0.00** | **0.00** | **0.00** | **0.00** |
| comb-14 | 14 | **0.00** | **0.00** | *25.98* | **0.00** | **0.00** |
| **rings-12** | 12 | 17.63 | 17.63 | *6.61* | *6.61* | *3.24* |
| **rings-14** | 14 | 8.58 | 8.58 | *26.06* | *3.82* | *1.52* |
| uniform-12 | 12 | 12.20 | **0.00** | **0.00** | *5.46* | **0.00** |
| uniform-14 | 14 | 30.36 | **0.00** | *15.32* | *1.47* | *1.47* |
| uniform-16 | 16 | 12.01 | 1.01 | *9.78* | *7.60* | *0.31* |

### 2.3 Counterexample to Exactness — `rings-12`

We exhibit an explicit 12-point Euclidean instance on which SiftTSP fails to recover the optimum at every tested depth. The instance, an inner ring of 6 points of radius $\approx 2$ and an outer ring of 6 points of radius $\approx 4$ both centred at $(5, 5)$, is materialised verbatim in `counterexample.py`.

**Table 2.** Counterexample `rings-12` — Held–Karp optimum $L^\star = 29.7823$.

| Algorithm | Length $L$ | Gap |
|---|---|---|
| Held–Karp (exact) | 29.7823 | — |
| SiftTSP, $d = 1$, $\tau = 6$ | 31.7503 | +6.61% |
| SiftTSP, $d = 2$, $\tau = 6$ | 31.7503 | +6.61% |
| SiftTSP, $d = 3$, $\tau = 6$ | 30.7478 | +3.24% |

**Proposition 1 (Non-exactness).** *SiftTSP with $\tau = 6$ and any $d \in \{1, 2, 3\}$ is not exact on the Euclidean TSP.*

*Proof.* The 12-point instance `RINGS_12` in `counterexample.py` is a witness: Held–Karp's optimum has length 29.7823, while SiftTSP at $d = 1, 2$ returns 31.7503 and at $d = 3$ returns 30.7478, all strictly greater. Values are reproducible by `python counterexample.py`. □

The following is the structural reason *why* `rings-12` defeats the algorithm. Proposition 1 above is empirical; Proposition 2 below is a structural impossibility result that applies to a broad class of algorithms including SiftTSP.

**Proposition 2 (Decomposition irrecoverability).** *Let $C \subset \mathbb{R}^2$ be a finite point set and let $\mathcal{D}$ be a finite family of partitions of $C$, each $D \in \mathcal{D}$ a partition $C = S_1^D \sqcup \cdots \sqcup S_{k_D}^D$ into pairwise disjoint components. Consider any algorithm $\mathcal{A}$ that, for each $D \in \mathcal{D}$, computes an open Hamiltonian path on each component of $D$ and then closes those paths into a tour by chaining them in some order with some orientation per path, and finally returns the shortest such candidate over all $D \in \mathcal{D}$ and all chainings. Call a tour $\pi$ on $C$ **component-contiguous** for $D$ if, for every component $S_i^D$, the cities of $S_i^D$ occupy a contiguous arc of $\pi$. If for every $D \in \mathcal{D}$ the global optimum $\pi^\star$ fails to be component-contiguous for $D$, then $L(\mathcal{A}(C)) > L(\pi^\star)$.*

*Proof.* For a fixed $D \in \mathcal{D}$ every candidate tour considered by $\mathcal{A}$ visits each component on a single contiguous arc, by construction (the open Hamiltonian path of a component is contiguous, and chaining concatenates components without interleaving them). Hence every candidate is component-contiguous for $D$. By hypothesis $\pi^\star$ is not component-contiguous for any $D \in \mathcal{D}$, so $\pi^\star$ is never among the candidates, and $\mathcal{A}(C)$ — the minimum over the candidates — has length strictly greater than $L(\pi^\star)$ unless some non-optimal candidate happens to have length exactly $L(\pi^\star)$. In the Euclidean generic position (no two distinct tours have equal length) this exception is empty. □

Proposition 2 applies to SiftTSP because the family $\mathcal{D}$ it explores (median bisections at depths $1, \ldots, d_{\max}$ along finitely many rotated axes and four axis-pairs) is *fixed in advance* and does not depend on $\pi^\star$. The `rings-12` Held–Karp optimum is empirically non-contiguous with respect to every $D$ tried at $d \leq 3$: any straight-line bisection either separates the two rings or cuts each ring into two diametrically opposed half-rings, while $\pi^\star$ alternates between inner and outer rings. Finer angular search adds resolution but does not change the decomposition primitive, so it cannot defeat the obstruction.

### 2.4 Observations

The following are observations from the 14-instance battery (Table 1). Each instance is run on a single fixed seed; we report descriptive findings, not statistical claims.

1. **Structured inputs are matched.** On zigzag, comb, and interleaved instances ($n \in \{12, 14, 16\}$), SiftTSP at $d \geq 2$ matches Held–Karp in every tested case, including instances where NN has gaps up to +38%.

2. **Two rotationally symmetric inputs are not matched, but 2-opt is beaten on both.** On `rings-12` and `rings-14`, NN and 2-opt are stuck at +17.63% and +8.58%; SiftTSP at $d = 3$ achieves +3.24% and +1.52%. Sample size is two; we do not claim this pattern holds on a wider population of rotationally symmetric instances.

3. **Increasing depth is not monotone.** On `uniform-12`, $d$=2 gives +5.46% while $d \in \{1, 3\}$ are exact. Finer subdivision can split clusters that the optimum traverses contiguously.

### 2.5 Discussion

**Where SiftTSP appears useful (on this battery).** Determinism (reproducible given $(C, d, \tau, m, s)$); strong performance on inputs with a clear geometric scan order (zigzag/comb/interleaved); and a suggestive regime on the two rotationally symmetric instances tested, where 2-opt is trapped. A larger study would be needed to claim this generalises.

**Limits.** Decomposition irrecoverability (§2.3) is the structural ceiling of *any* algorithm that explores only a fixed-in-advance family of geometric decompositions and then optimally chains the resulting component paths. The `rings-12` +3.24% wall is unmoved by the mill+sieve refinement, consistent with the obstruction being structural rather than angular-resolution-limited. The connection step's $(2^d - 1)! \cdot 2^{2^d - 1}$ growth blocks $d \geq 4$ unless replaced by a smarter chaining primitive.

**Open directions.**

- *Adaptive decomposition* (axes that depend on local geometry).
- *Recursive connection* (replace the brute-force chain by a recursive SiftTSP application on the path endpoints).

- *Hybridisation* with 2-opt or Lin–Kernighan [10] (LKH [11]) using SiftTSP as an initial-tour generator.
- *Characterisation of exactness regimes* — a sufficient geometric condition on $C$ guaranteeing exactness would clarify the value of the algorithm as a conditional exact solver.

None of these are claimed to defeat the decomposition-irrecoverability obstruction. SiftTSP is and will remain a heuristic; this work makes **no claim** of resolving $\mathcal{P}$ versus $\mathcal{NP}$.

## Code and Data Availability

The reference implementation, all instance families, and the verification scripts are available at https://github.com/McNopper/SiftTSP. Specifically: `SiftTSP.py` (algorithm), `tests/adversarial.py` (Table 1 battery, fixed seeds as above), `counterexample.py` (Table 2 `rings-12` instance with literal coordinates and Held–Karp verification). Numbers reported in this paper were produced by these scripts under CPython 3.11.

## References

## References


[1] S. A. Cook, "The complexity of theorem-proving procedures," in *Proceedings of the 3rd Annual ACM Symposium on Theory of Computing (STOC)*, pp. 151–158. ACM, New York (1971). https://doi.org/10.1145/800157.805047

[2] L. A. Levin, "Universal sequential search problems," *Problems of Information Transmission* **9**(3), 265–266 (1973).

[3] S. Aaronson, "P vs. NP," in R. Downey (ed.) *Fundamentals of Computation Theory*. Springer, Berlin (2009). https://www.scottaaronson.com/papers/pnp.pdf

[4] D. L. Applegate, R. E. Bixby, V. Chvátal, and W. J. Cook, *The Traveling Salesman Problem: A Computational Study*. Princeton University Press, Princeton (2006).

[5] R. M. Karp, "Reducibility among combinatorial problems," in R. E. Miller and J. W. Thatcher (eds.) *Complexity of Computer Computations*, pp. 85–103. Plenum Press, New York (1972). https://doi.org/10.1007/978-1-4684-2001-2_9

[6] C. H. Papadimitriou, "The Euclidean travelling salesman problem is NP-complete," *Theoretical Computer Science* **4**(3), 237–244 (1977). https://doi.org/10.1016/0304-3975(77)90012-3

[7] M. Held and R. M. Karp, "A dynamic programming approach to sequencing problems," *Journal of the Society for Industrial and Applied Mathematics* **10**(1), 196–210 (1962). https://doi.org/10.1137/0110015

[8] R. M. Karp, "Probabilistic analysis of partitioning algorithms for the traveling-salesman problem in the plane," *Mathematics of Operations Research* **2**(3), 209–224 (1977). https://doi.org/10.1287/moor.2.3.209

[9] N. Christofides, "Worst-case analysis of a new heuristic for the travelling salesman problem," Report 388, Graduate School of Industrial Administration, Carnegie Mellon University (1976).

[10] S. Lin and B. W. Kernighan, "An effective heuristic algorithm for the traveling-salesman problem," *Operations Research* **21**(2), 498–516 (1973). https://doi.org/10.1287/opre.21.2.498

[11] K. Helsgaun, "An effective implementation of the Lin–Kernighan traveling salesman heuristic," *European Journal of Operational Research* **126**(1), 106–130 (2000). https://doi.org/10.1016/S0377-2217(99)00284-2

[12] S. Arora, "Polynomial time approximation schemes for Euclidean traveling salesman and other geometric problems," *Journal of the ACM* **45**(5), 753–782 (1998). https://doi.org/10.1145/290179.290180

[13] J. L. Bentley, "Fast algorithms for geometric traveling salesman problems," *ORSA Journal on Computing* **4**(4), 387–411 (1992). https://doi.org/10.1287/ijoc.4.4.387